

■ Mojito Builder

Documentation Technique

Application Web Interactive — HTML · CSS · JavaScript

HTML

CSS

JavaScript

Fetch API

localStorage

Projet pédagogique — JavaScript Moderne

1. Présentation du projet

Mojito Builder est une application web développée en **HTML**, **CSS** et **JavaScript** permettant à l'utilisateur de créer un mojito personnalisé.

L'utilisateur peut :

- Choisir les ingrédients
- Calculer automatiquement le prix
- Sauvegarder ses mojitos
- Consulter des recettes via une API REST
- Stocker des données dans le navigateur

2. Technologies utilisées

Technologie	Rôle
HTML	Structure des pages
CSS	Mise en forme et design
JavaScript	Logique de l'application
Fetch API	Communication avec API REST
localStorage	Sauvegarde permanente
sessionStorage	Gestion de session utilisateur

3. Structure du projet

Fichier	Description
■ index.html	Pages, formulaires, zones d'affichage
■ style.css	Design, animations, responsive
■■ script.js	Logique, événements, DOM, API

4. Description des fichiers

■ index.html

Contient :

- Page de connexion
- Formulaire de création du mojito
- Zone d'affichage des résultats

- Liste des mojitos sauvegardés
- Section API

■ style.css

Contient :

- Design général
- Animations
- Couleurs
- Responsive design
- Style des cartes et boutons

■■ script.js

Contient :

- Logique JavaScript
- Événements
- Calcul des prix
- Stockage des données
- Appels API
- Manipulations du DOM

5. Fonctionnalités principales

■ Système de connexion

L'utilisateur saisit son **nom** et son **email**. Les informations sont stockées dans **sessionStorage**.

Après connexion, le message s'affiche automatiquement :

```
Welcome NomUtilisateur ■
```

■ Création d'un Mojito

L'utilisateur peut choisir :

- Le parfum principal
- Le fruit
- Le niveau de sucre
- Le nombre de glaçons

■ Calcul dynamique du prix

Le prix est calculé automatiquement selon le parfum, le fruit et les glaçons :

```
Prix total = prix parfum + prix fruit + (nombre glaçons × 0.5)
```

■ Validation des données

Le système vérifie que tous les champs sont remplis et que les données sont valides :

```
if(flavor === ""){ alert("Please complete all fields"); }
```

■ Compatibilité des ingrédients

Le système analyse la compatibilité entre le parfum et le fruit. Exemple :

```
Mint + Kiwi → Excellent ■
```

■ Sauvegarde des Mojitos

Les mojitos créés sont sauvegardés avec **localStorage**. Les données restent disponibles même après fermeture du navigateur.

■ API REST avec fetch()

L'application récupère des recettes depuis **TheCocktailDB API** en utilisant **fetch()**, **async/await**, **Promise** et **JSON**.

6. Manipulation du DOM

Le projet utilise :

- **getElementById()**
- **createElement()**

- `appendChild()`
- `innerHTML`
- `addEventListener()`

Exemple :

```
const card = document.createElement("div");
```

7. Gestion des événements

Les événements utilisés :

- `click`
- `load`
- `change`

Exemple :

```
button.addEventListener("click", createMojito);
```

8. Structures de données

Tableaux (Arrays)

```
const fruits = [ "Kiwi", "Mango" ];
```

Objets (Objects)

```
const flavorPrices = { Mint: 8, Lemon: 7 };
```

9. Gestion asynchrone

Le projet utilise **async/await** pour gérer les requêtes API de manière asynchrone :

```
const response = await fetch(url);
```

10. Objectif pédagogique

Ce projet permet de pratiquer :

- Le JavaScript moderne
- Le DOM
- Les événements
- Le stockage local
- Les API REST
- La programmation asynchrone
- L'organisation du code

11. Conclusion

Le projet **Mojito Builder** est une application complète permettant de démontrer la maîtrise des concepts fondamentaux du JavaScript moderne à travers une interface interactive et dynamique.